

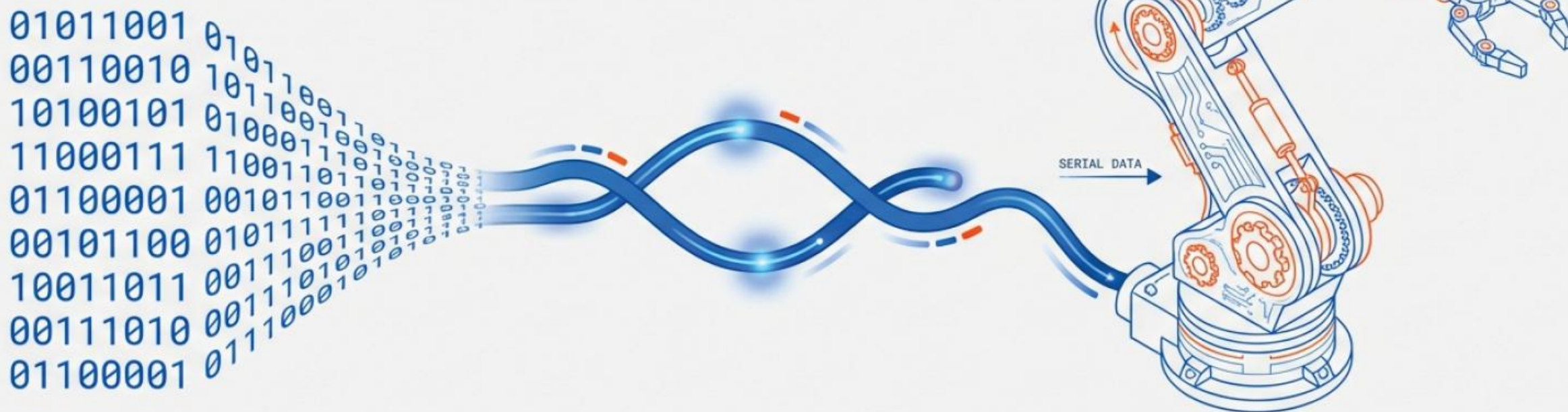
Robótica Industrial

La robótica combina la ciencia y la tecnología para crear máquinas inteligentes capaces de interactuar con el mundo.



Del Bit al Robot: Dominando la Comunicación Serial

El Puente Vital entre el Hardware (Arduino) y el Software de Control



Desde encender un LED hasta la telemetría compleja con ROS:
todo comienza con un bit viajando por un cable.

La "Fila India" de los Datos

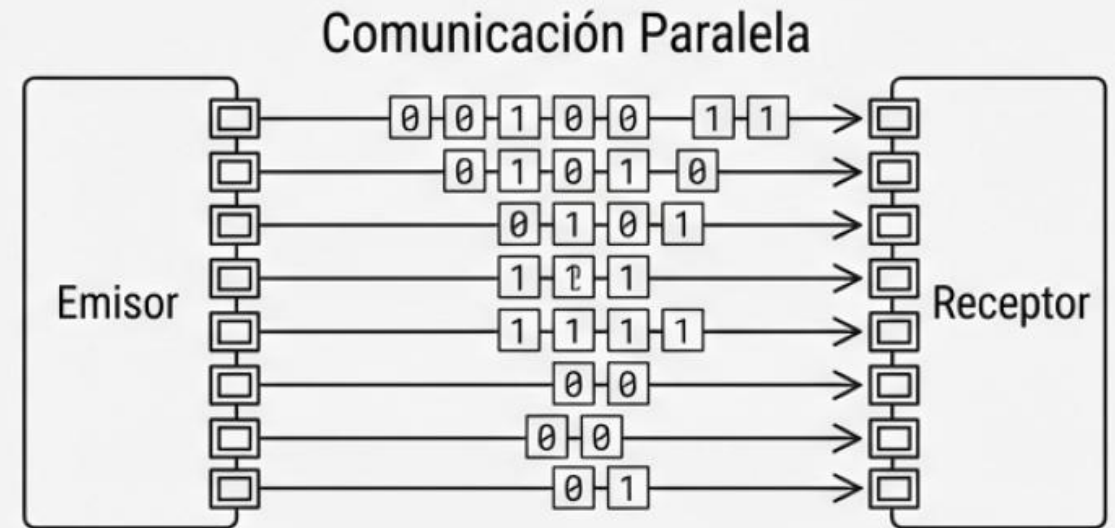
Definición: La comunicación serial transmite datos secuencialmente, bit por bit, a través de una sola línea o canal.

Analogía: Imagine una entrada estrecha donde las personas (bits) deben pasar una por una en "fila india", en lugar de entrar todas juntas de la mano (paralelo).

Por qué Arduino la usa:

1. Reduce la cantidad de pines necesarios.
2. Es el estándar industrial para conectar microcontroladores con PCs.

Paralelo vs. Serial



El Hardware Detrás de la Magia: UART y USB

UART (Universal Asynchronous Receiver-Transmitter):

El periférico de hardware dentro del chip Arduino que gestiona el envío y recepción de bits.

Pines Clave:

- **TX (Transmit):** Salida de datos.
- **RX (Receive):** Entrada de datos.

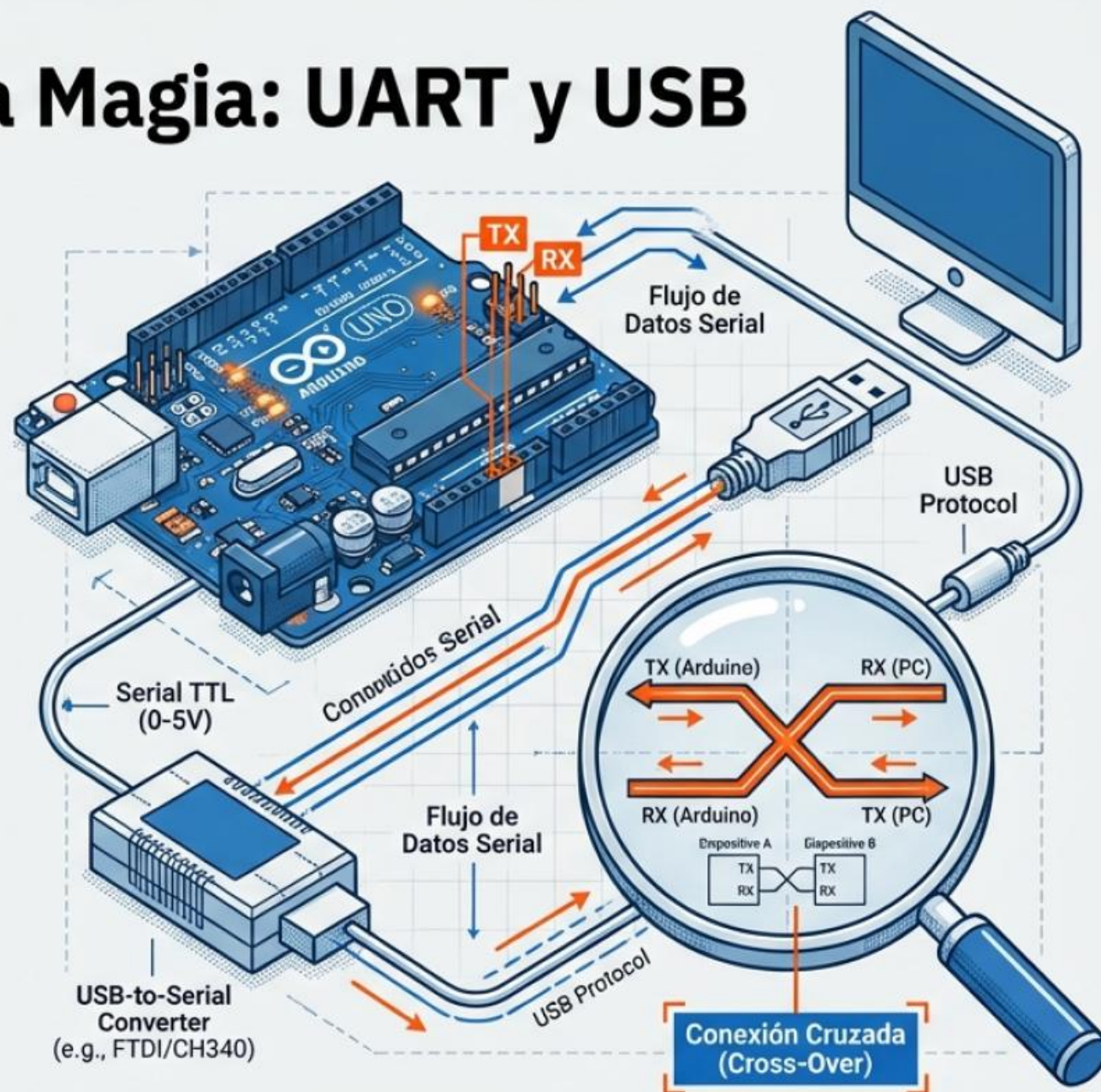
Regla de Oro: El TX de uno va al RX del otro (conexión cruzada).

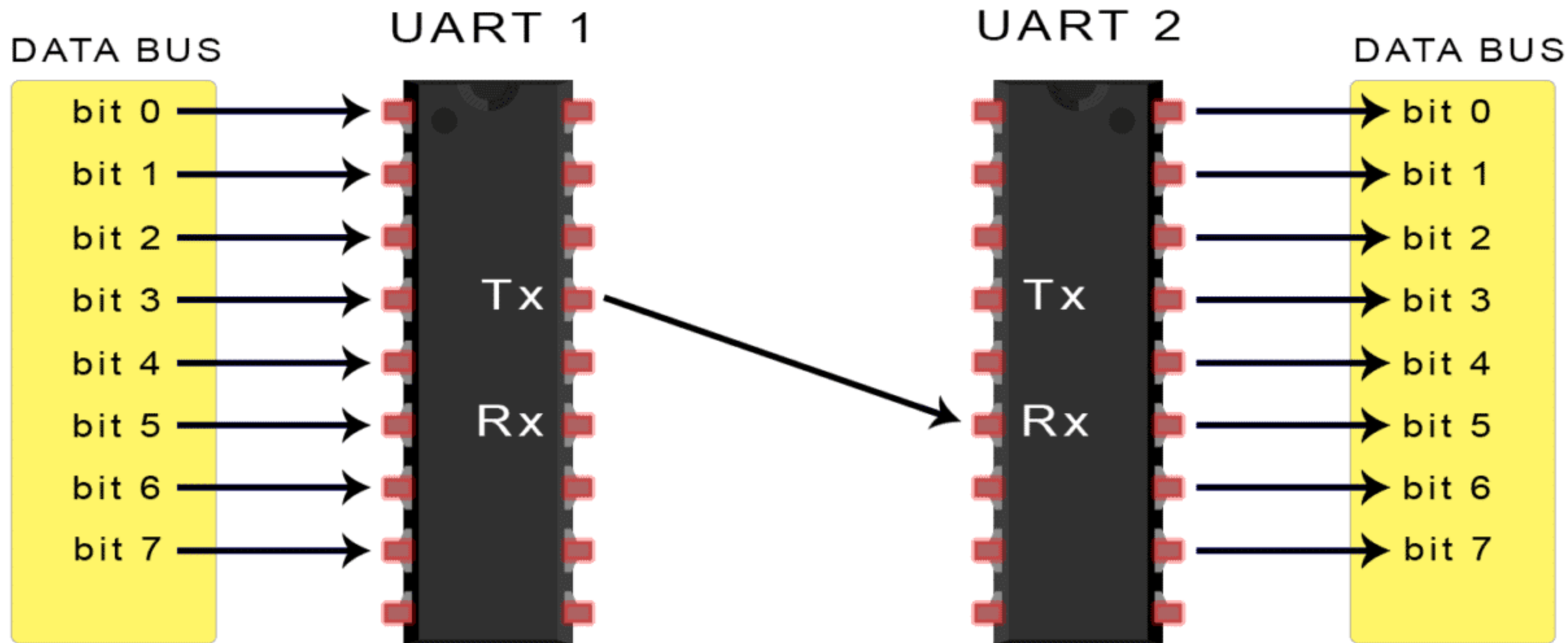
El Traductor USB:

Chips como FTDI o CH340 convierten la señal serial del Arduino al protocolo USB de la PC.

Niveles de Voltaje:

Arduino usa TTL (0-5V). Precaución: RS-232 usa +/-12V y requiere convertidores (MAX232) para no quemar el chip.

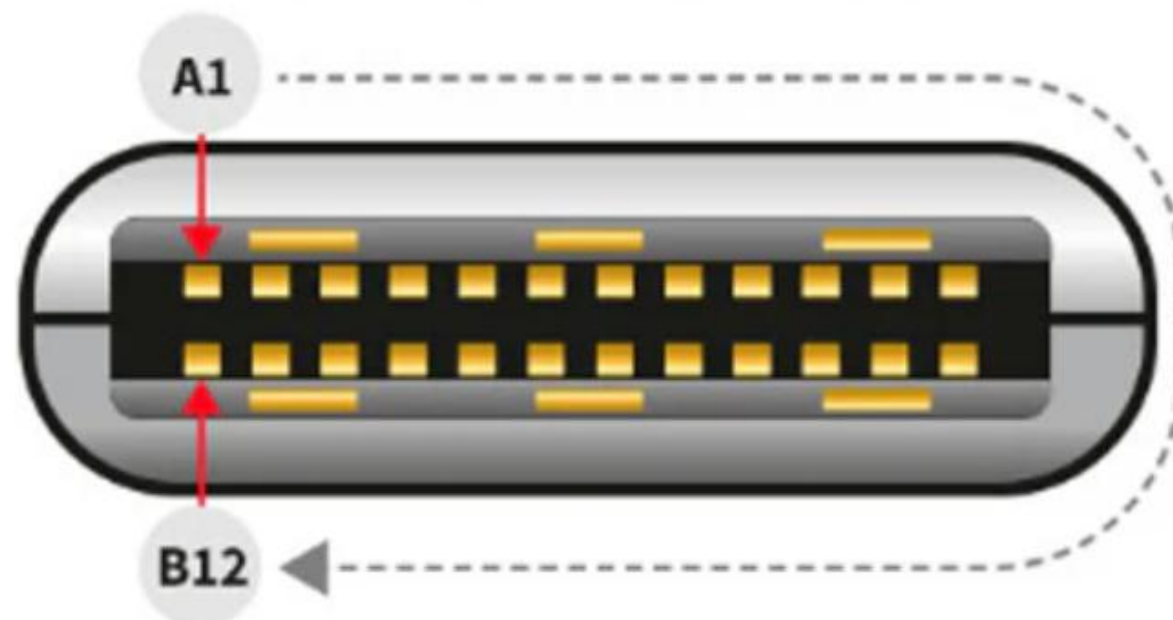




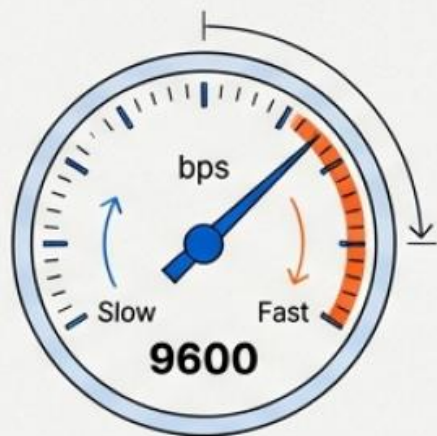


A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	A11	A12
GND	TX1+	TX1-	V _{BUS}	CC1	D+	D-	SBU1	V _{BUS}	RX2-	RX2+	GND
GND	RX1+	RX1-	V _{BUS}	SBU2	D-	D+	CC2	V _{BUS}	TX2-	TX2+	GND
B12	B11	B10	B9	B8	B7	B6	B5	B4	B3	B2	B1

	Mass (ground)
	High speed data path (1)
	High speed data path (2)
	Power supply
	Pins to recognize the side of the male
	Overvoltage protection
	USB 2.0

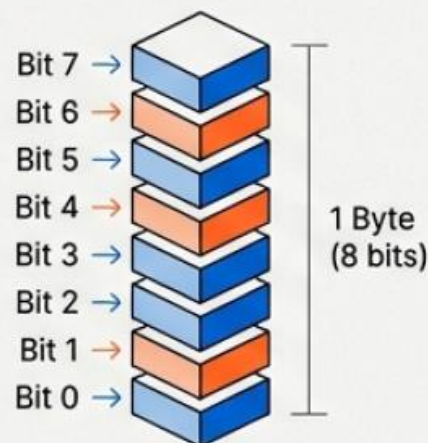


El Lenguaje de los Datos: Bits, Bytes y Baudios



Baud Rate (Velocidad)

La velocidad de la "conversación". Ambos dispositivos deben estar sincronizados (ej. 9600 bps). Si uno habla rápido y el otro escucha lento, el mensaje se pierde.



El Byte

La unidad estándar de 8 bits.

Valor máximo (unsigned 8-bit): **255**.

Reto: Muchos sensores usan valores de 16 bits (0-4095).



Binario Básico

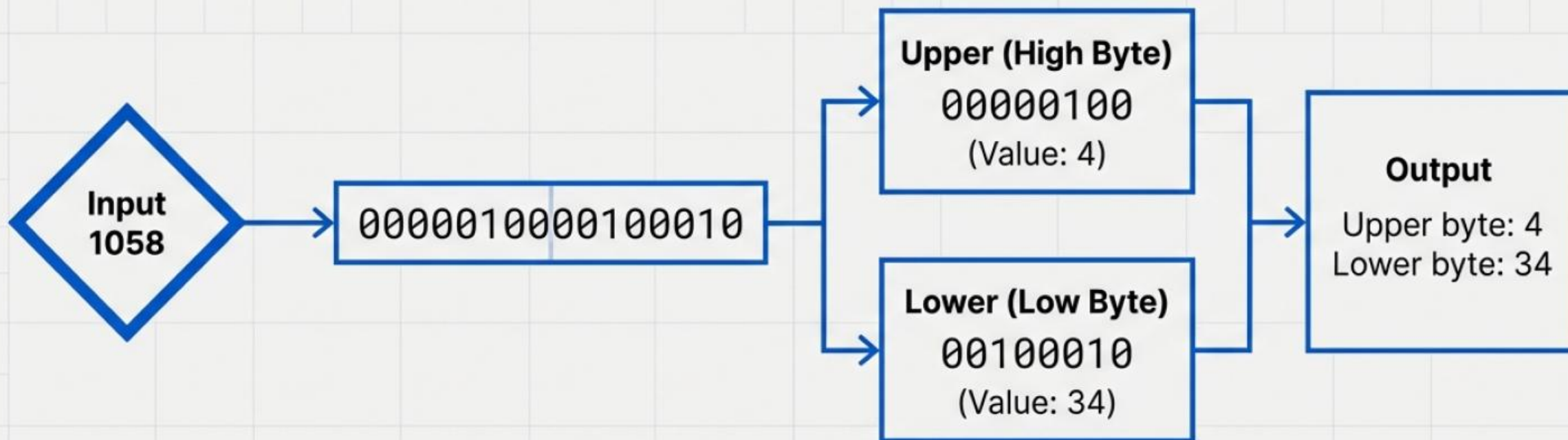
$$\begin{array}{ccc} 1011 & = & 11 \\ \text{(Binario)} & & \text{(Decimal)} \end{array}$$

$$2^3 + 2^1 + 2^0$$

El Reto Técnico: Enviando Números Grandes (16-bit)

El Problema: El puerto serial envía paquetes de 8 bits (máx 255). ¿Cómo enviamos el valor 1058?

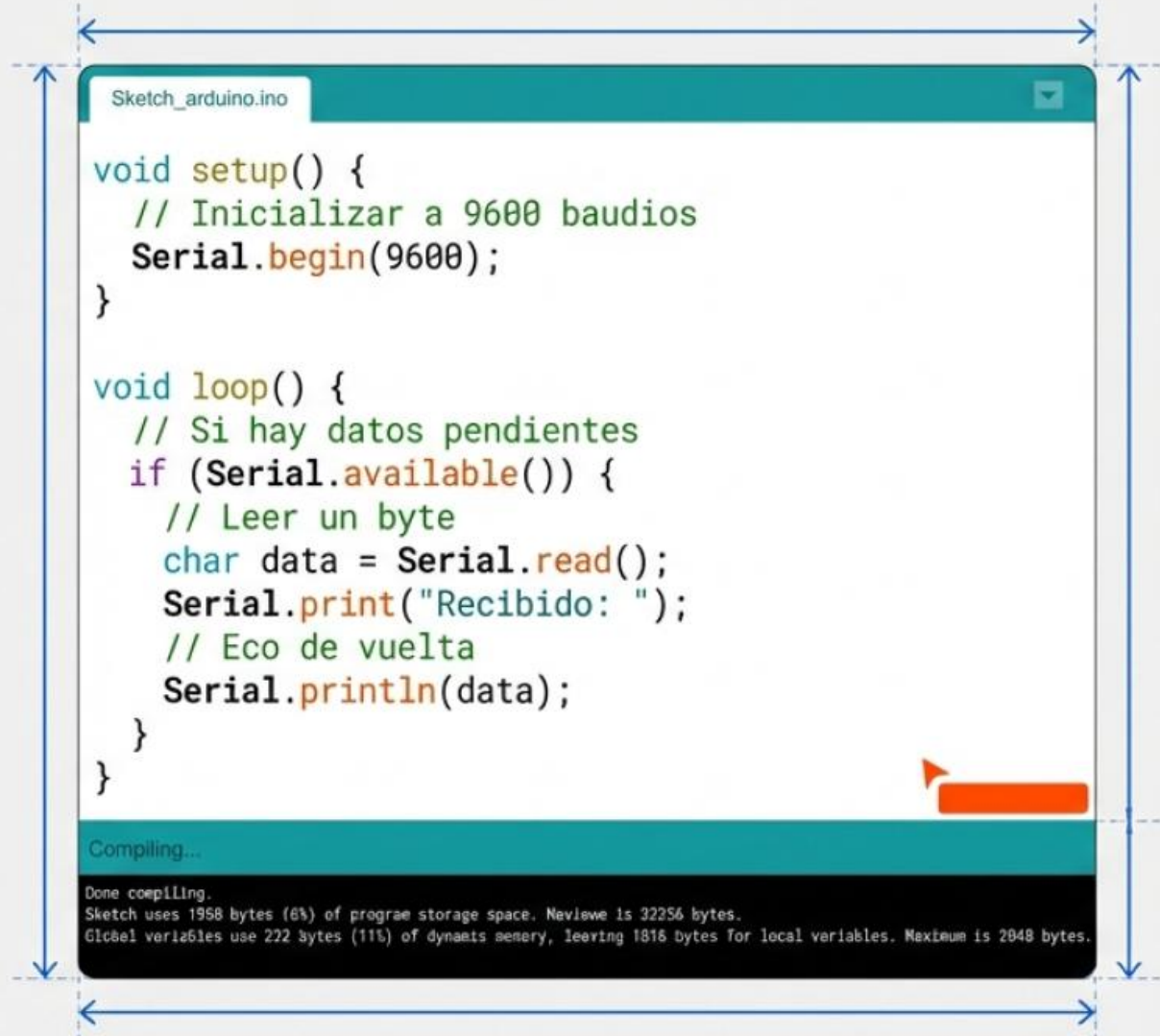
La Solución (Bit-shifting): Dividir el entero de 16 bits en dos bytes (High Byte y Low Byte), enviarlos por separado y reconstruirlos.



Implementación en Arduino (C++)

Funciones Clave:

- **Serial.begin(speed)**
Inicializa el puerto. La velocidad (*speed*) debe coincidir con el receptor.
- **Serial.print(val)**
Envía datos como texto legible (ASCII) a la PC.
- **Serial.write(val)**
Envía el byte crudo (valor binario).
- **Serial.read()**
Lee el byte entrante.



The image shows a screenshot of the Arduino IDE interface. At the top, a tab is labeled "Sketch_arduino.ino". The main text area contains the following C++ code:

```
void setup() {  
  // Inicializar a 9600 baudios  
  Serial.begin(9600);  
}  
  
void loop() {  
  // Si hay datos pendientes  
  if (Serial.available()) {  
    // Leer un byte  
    char data = Serial.read();  
    Serial.print("Recibido: ");  
    // Eco de vuelta  
    Serial.println(data);  
  }  
}
```

Below the code editor, there is a status bar. It shows "Compiling..." in a teal bar, followed by a black bar with white text: "Done compiling. Sketch uses 1958 bytes (6%) of program storage space. Maximum is 32256 bytes. Global variables use 222 bytes (11%) of dynamic memory, leaving 1816 bytes for local variables. Maximum is 2048 bytes." There is also a small orange arrow icon pointing right.